Unit - 3

CHAPTER 3

Relational Data Model

Contents

- Introduction
- Relational Model Concepts
- Constraints on Relations
- Categories of Constraints
- Domain Constraints
- Key Constraints and Constraints on Null Values
- Entity Integrity, Referential Integrity and Foreign Keys
- Integrity Constraints Over Relations
- Operations on Relations
- Relational Algebra
- Relational Algebra Operations from Set Theory
- Binary Relational Operations
- Additional Relational Operations
- Examples of Queries in Relational Algebra
- Review Questions

Scanned with OKEN Scanner

3.1 Introduction

Relational model in DBMS In relational model, the data and relationships are represented by collection of inter-related tables. Each table is a group of column and rows, where column represents attribute of an entity and rows represents records. The table name and column names are helpful to interpret the meaning of values in each row. In the formal relational model terminology, a row is called a tuple, a column header is called an attribute, and the table is called a relation. The data type describing the types of values that can appear in each column is represented by a domain of possible values.

3.2 Relational Model Concepts

Domains

A domain is a unique set of values permitted for an attribute in a table. Domain is atomic. A data type or format is also specified for each domain. It is possible for several attributes to have the same domain. The data type for Student_age is an integer number.

Example :

- A domain of month-of-year can accept January, February to December as possible values.
- A domain of integers can accept whole numbers that are negative, positive and zero,.
- A domain of age attribute can only be a positive integer. Some examples of domains follow:
- A domain of mobile numbers: The set of ten digit positive numbers.
- A domain of Aadhar number : The set of valid 12 digit positive numbers.

Attribute

In Relational DBMS, a table organizes data in rows and columns. The columns are known as attributes whereas the rows are known as records.

Example: We define the roll_no of the 'Student' by a numeric value. Here, roll_no is an attribute. The permitted values are only integers and hence, 'integer' is the domain of attribute 'roll_no'. Each attribute is represented by a separate column in a relational table.

Example : The entity 'Student' has properties like Name, Address, Roll_no, Mobile_no, Age, DOB, Class, Section, etc. These properties are called as attributes of Student entity.

Tuple

The term tuple is often also called row and record in a database table. It is nothing but a single row of a table, which contains a single record. Generally, the database table is viewed as a collection of tuples. A tuple gives particular information of the relation (table).

Example: Let us consider an employee table, we need to store the details of 10 employees then the database table will consist of 10 tuples every each tuple contains details of that particular employee i.e 1 tuple for storing the record of one employee.

Example : In customer table, one row gives information about one customer only. In student table, one row gives information about one student only.

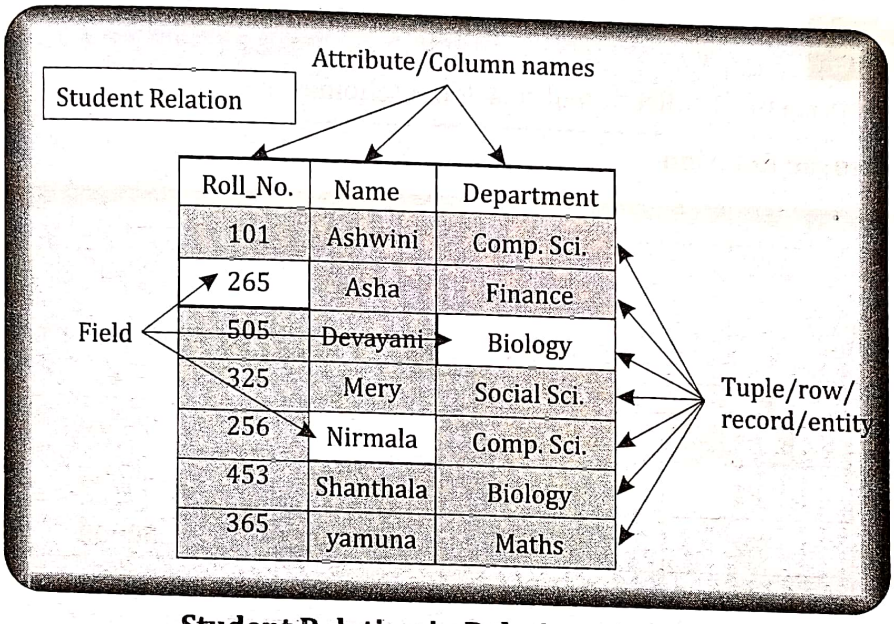
Scanned with OKEN Scanner

Relation

Relation is nothing but a table in relational DBMS. Everything in a relational database is stored in the form of tables. The RDBMS database uses tables to store data. A table is a collection of related data entries and contains rows and columns to store data. A relation is nothing but a table of values. A table has two properties rows and columns. Every row in the table represents a collection of related data values. These rows in the table denote a real-world entity or relationship. The columns are the table's attributes, and the rows represent the data records. A single row is known as a tuple.

Each table represents some real-world objects such as person, place, or event about which information is collected. The organized collection of data into a relational table is known as the **logical view of the database.**

Example **Student Relation**



Student Relation in Relational Model

Relation Schema

A relation schema represents the name of the relation with its attributes. A relational schema is the design for the table. It includes none of the actual data, but is like a blueprint or design for the table, so describes what columns are on the table and the data types. It may show basic table constraints (Example, if a column can be null) but not how it relates to other tables.

A relation schema R, denoted by $R(A_1,A_2, ..., A_n)$, is made up of a relation name R and a list of attributes, $A_1,A_2, ...,A_n$. Each attribute A_i is the name of a role played by some domain D in the relation schema R. D is called the domain of A_i and is denoted by $dom(A_i)$. The relation schema $R(A_1,A_2, ...,A_n)$, also denoted by r(R), is a set of n -tuples $r = \{t_1,t_2, ...,t_n\}$.

Degree and Cardinality

- The number of attributes or columns in a relation is called the Degree of the relation.
- The number of tuples/ rows in a relation is called the Cardinality of the relation.

Scanned with OKEN Scanner

Relation Instance and Relation State

Relation Instance is a finite set of tuples in the relational DBMS system. Relation instances never have duplicate tuples. Relational instance at a particular moment in time (a set of tuples) is called as **Relation State.**

Relational Database Schema:

Relational Database Schema contains a set of relational schemas and integrity constraints such as entity integrity constraint, referential integrity constraint, key and domain constraint. Data Definition Language (DDL) helps us to describe the relational database schema.

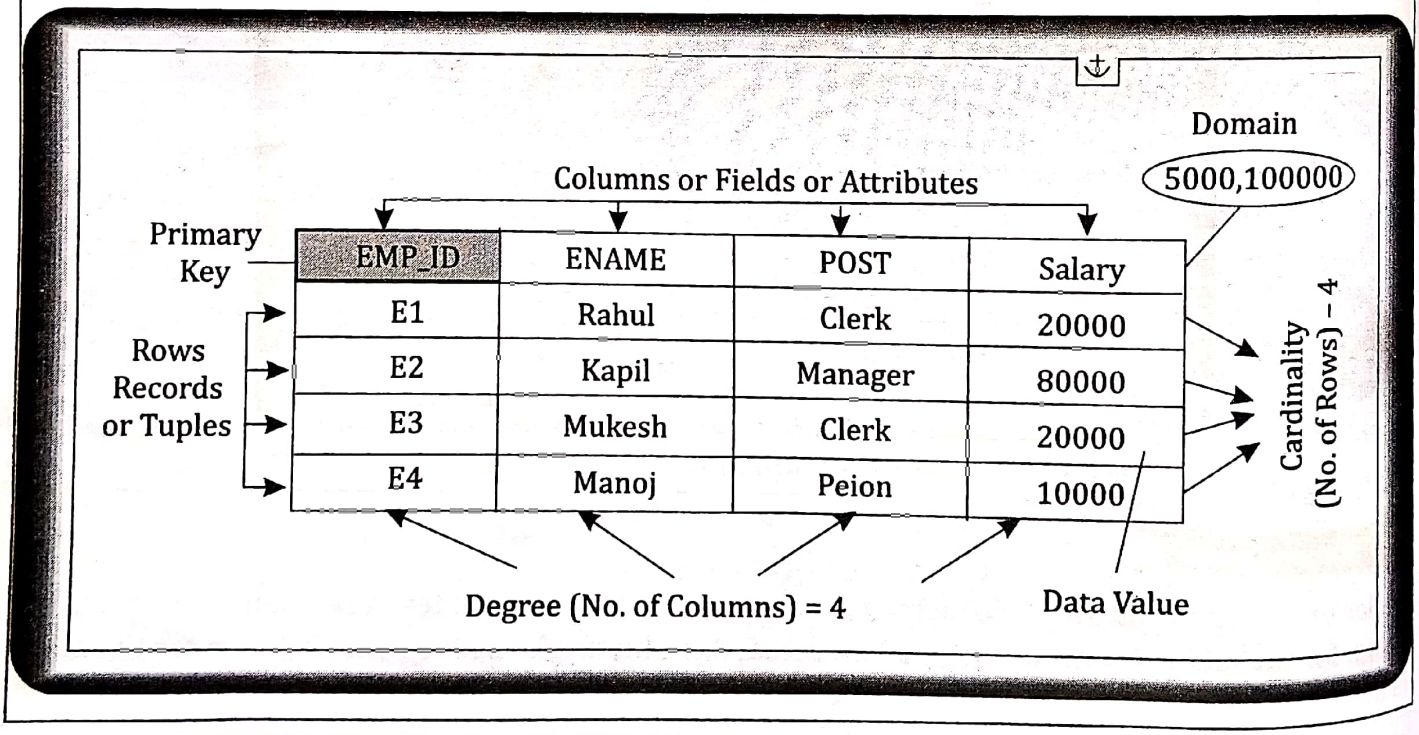
Relational Database State:

A set of relational schema states at a particular moment in time is called as Relational Database State.

Relational Model:

Relational Model represents a collection of relational schemas and constraints.

Example **Employee Relation**



3.2.1 Characteristics of Relations

Ordering of Tuples in a Relation

A relation is defined as a set of tuples. The tuples in a relation do not have any particular order. In other words, a relation is not sensitive to the ordering of tuples. However, in a file, there is always an order among the records. This ordering indicates first, second, third, i^{th}.... and last records in the file. Tuple ordering is not part of a relation definition because a relation attempts to represent facts at a logical or abstract level. Many tuple orders can be specified on the same relation.

Scanned with OKEN Scanner

For example, tuples in the EMPLYOEE relation could be ordered by values of EID, ENAME , EDEPT or some other attribute. The definition of a relation does not specify any order: There is no preference for one ordering over another.

Ordering of Attributes in a Relation

The ordering of attributes is not important, because the attribute name appears with its value. There is no reason to prefer having one attribute value appear before another in a tuple. When a relation is implemented as a file, the attributes and the values within tuples are ordered.

Values in a Tuple

All values are considered atomic. A special null value is used to represent values that are unknown or inapplicable to certain tuples. In general, NULL values, means value unknown or value exists but is not available.

ROLL_NO.	NAME	AGE	PHONE_NO	ADDRESS
101	Anusha	22	9687466166	Basavangudi
105	Namitha	20	6985634586	Jayanagar
104	Rajesh	21	9874561235	Vijayanagar
106	Pooja	24	9658745698	Hanumanthnagar

Interpretation (Meaning) of a Relation

The relation schema can be interpreted as a declaration or a type of assertion. Each tuple in the relation can then be interpreted as a fact or a particular instance of the assertion.

For example, the first tuple in the above table asserts the fact that there is a STUDENT whose NAME is Anusha, ROLL_NO is 101, Age is 22, and so on.

3.2.2 Relational Model Notations

The following are the notations used in Relational Model:

- A relation schema R of degree n is denoted by $R(A_1, A_2, ..., A_n)$.
- The uppercase letters Q, R, S denote relation names.
- The lowercase letters q, r, s denote relation states.
- The letters t, u, v denote tuples.
- In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation whereas STUDENT (ROLL_NO,NAME..... ...) refers only to the relation schema.
- An attribute A can be qualified with the relation name R to which it belongs by using the dot notation R.A—for example, STUDENT.Name or STUDENT.Age. This is because the same name may be used for two attributes in different relations. However, all attribute names in a particular relation must be distinct.
- An n-tuple t in a relation r(R) is denoted by $t = <v_1, v_2, v_3, v_4 ..... v_i>$, where v_i is the value corresponding to attribute A_i. The following notation refers to component values of tuples:

Scanned with OKEN Scanner

- ✓ Both $t[A_i]$ and $t.A_i$ (and sometimes t[i]) refer to the value v_i in t for attribute A_i)
- ✓ **Example :** Consider the tuple t = <101 , ' Anusha ' , 22 , 9687466166 , Basavangudi > from the STUDENT relation; we have t[ROLL_NO] = 101, and t[NAME, AGE, PHONE_NO]=< 'Anusha' , 22 ,9687466166>

3.3 Constraints on Relations

In a relational database, there will be typically many relations, and the tuples in those relations are usually related in various ways. The state of the whole database will correspond to the states of all its relations at a particular point in time. There are generally many restrictions or constraints on the actual values in a database state.

On modeling the design of the relational database we can put some restrictions like what values are allowed to be inserted in the relation, what kind of modifications and deletions are allowed in the relation. Constraints restrict the data that can be stored in relations. These are usually defined using expressions that result in a boolean value, indicating whether or not the data satisfies the constraint. Constraints can apply to single attributes, to a tuple (restricting combinations of attributes) or to an entire relation.

Definition: Constraints on Relations

A property assigned to a table column that prevents certain types of invalid data values from being placed in the column.

3.4 Categories of Constraints

The various restrictions on data that can be specified on a relational database in the form of constraints can be generally classified into four main categories:

1. Inherent Model Based Constraints.
2. Schema Based Constraints
3. Application Based Constraints.
4. Data Dependencies.

3.4.1 Inherent Model Based Constraints

Constraints that are inherent in the data model are called as **Inherent Model Based Constraints**. The characteristic of relations that discussed in previous section is an example of inherent constraints of the relational model.

Example: The constraint that a relation cannot have duplicate tuples is an inherent constraint.

3.4.2 Schema Based Constraints

Constraints that can be directly expressed in the schemas of the data model, typically by specifying them in the DDL (Data Definition Language), are called as **Schema Based Constraints**. Different types of constraints that can be expressed in the relational model. The schema based constraints are as follows:

- Domain Constraints,
- Constraints on Nulls,
- Referential Integrity Constraints
- Key Constraints,
- Entity Integrity Constraints

Scanned with OKEN Scanner

3.4.3 Application Based Constraints

Constraints in this category are more general and are difficult to express and enforce within the data model, so they are usually checked within application programs. Therefore Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs are called as **Application Based Schema constraints.**

Definition: Application Based Constraints

Constraints that cannot be directly expressed in the schemas of the data model, and hence must be expressed and enforced by the application programs.

3.4.4 Data Dependencies Constraints

Another important category of constraints is *data dependencies,* which include *functional dependencies* and *multivalued dependencies.* They are used mainly for testing the "goodness" of the design of a relational database and are utilized in a process of *normalization.*

3.5 Domain Constraints

A domain is defined as the set of all unique values permitted for an attribute. Therefore domain refers to a set of possible values for each attribute associated with an entity set. Domain constraints in the schema specify an important condition that the user wants each instance of the relation to satisfy. Domain constraints specify that within each tuple, the value of each attribute A must be an atomic value from the domain dom(A).

Example

A domain of date is the set of all possible valid dates, a domain of integer is all possible whole numbers, a domain of day-of-week is Monday, Tuesday ... Sunday. This in effect is defining rules for a particular attribute. If it is determined that an attribute is a date then it should be implemented in the database to prevent invalid dates being entered.

A classic example of this is where the data from a legacy system is loaded into a newly designed database. The new system is well designed. Columns that hold dates are defined as ***date type*** whereas, in the old system, they were defined as character strings. Data may be rejected because of invalid dates, e.g. 30 February 2000.

The data types associated with domains typically include standard data types:

- Numeric data types for integers and real numbers
- Characters
- Fixed-length strings
- Date, time, timestamp
- Other special data types
- Booleans
- Variable-length strings
- Money

Scanned with OKEN Scanner

Definition: Domain Constraint

Domain constraints in the schema specify an important condition that the user wants each instance of the relation to satisfy. Domain integrity helps to ensure the length and type of data, the null value, the allowable value and the default value.

3.6 Key Constraints and Constraints on Null Values

3.6.1 Key Constraints

In the relational model, keys are important because they are used to ensure that each row in a table is uniquely identifiable. They are also used to establish relationships among tables and to ensure the integrity of the data. Therefore, a proper understanding of the concept and use of keys in the relational model is very important. A key consists of one or more attributes that determine other attributes. For example, an Student ID identifies all of the Student attributes, such as the student name, age, date of birth, course, and semester.

The key's role is based on a concept known as **determination**. In the context of a database table, the statement "A determines B" indicates that if we know the value of attribute A, we can look up (determine) the value of attribute B. For example, knowing the Student ID in the STUDENT table means that we are able to look up (determine) that student's name, age, date of birth, course, semester and so on.

Key Constraints are called uniqueness constraints since it ensures that every tuple in the relation should be unique.

Why we need a Key?

- Keys helps to identify any row of data in a table.
- In a real-world application, a table could contain thousands of records. Moreover, the records could be duplicated. Keys in relational DBMS (RDBMS) ensure that we can uniquely identify a table record despite these challenges.
- Allows to establish a relationship between between tables
- Helps to enforce identity and integrity in the relationship.

Different Types of Keys in Relational Model

A relation is defined as a set of tuples. By definition, all elements of a set are distinct hence; all tuples *in a relation must also be distinct. This means that no two tuples can have the same combination of values for all their attributes.* Usually, there are other subsets of attributes of a relation schema R with the property that no two tuples in any relation state r of R should have the same combination of values for these attributes.

1. Super Key

Superkey is an attribute or set of attributes which uniquely identifies each tuple in a relation. Every relation has at least 1 default super key as the set of all attributes.

Scanned with OKEN Scanner

Example : The ID attribute of the relation STUDENT is sufficient to distinguish one student tuple fromanother. Thus, ID is a superkey. The name attribute of student, on the other hand, is not a superkey, because several students might have the same name.

Suppose that we denote a subset of attributes by SK then for any two *distinct* tuples t1 and *t2* in a relation state r of R, we have the constraint that

$$t1[SK] \neq t2[SK]$$

Any such set of attributes SK is called a **superkey (SK)** of the relation schema R.

A superkey SK specifies a *uniqueness constraint* that no two distinct tuples in any state r of R can have the same value for SK. Every relation has at least one default superkey (the set of all its attributes). A superkey can have redundant attributes, however, so a more useful concept is that of a **key**, which has no redundancy.

Example : In the STUDENT table, the superkey could be any of the following:
STUDENT_ID}, {AadharNo}, {STUDENT_ID, AadharNo}, {STUDENT_ID, AadharNo, Age}, {AadharNo,Name,Course}, {AadharNo, Age, Course}, {STUDENT_ID, AadharNo, Age, Course}, {AadharNo,Name,Course, Age} and so on.

In fact, {STUDENT_ID} or {AadharNo}, with or without additional attributes, can be a superkey even when the additional attributes are redundant.

2. Candidate Key

A Candidate key is a minimal super key with no redundant attributes (or) The minimal set of attributes that can uniquely identify a tuple is known as a candidate key.

Example : Let us consider the Student Relation contains the below attributes.

{STUDENT_ID, AadharNo, Name, Age, Course}.

Superkeys are : {STUDENT_ID}, {AadharNo}, {STUDENT_ID, AadharNo}, {STUDENT_ID, AadharNo, Age}, {AadharNo,Name,Course}, {AadharNo, Age, Course}, {STUDENT_ID, AadharNo, Age, Course}, {AadharNo,Name,Course, Age} and so on.

The minimal super keys (candidate keys) are : {STUDENT_ID} and {AadharNo}

Only these two sets are candidate keys as all other sets are having redundant attributes that are not necessary for unique identification.

Properties of Candidate key:

- It is a minimal super key.
- It is a super key with no extra or redundant attributes.
- The minimal set of attributes that can uniquely identify a record.
- It must contain unique values.
- It can contain NULL values.
- Every table must have at least a single candidate key.

Scanned with OKEN Scanner

- A table can have multiple candidate keys but only one primary key. (the primary key cannot have a NULL value, so the candidate key with NULL value can't be the primary key).
- There can be more than one candidate key in a relation.

Super key vs Candidate Key:

1. First we have to understand that all the candidate keys are super keys. This is because the candidate keys are chosen out of the super keys.
2. How we choose candidate keys from the set of super keys?

   We look for those keys from which we cannot remove any fields. In the above example, we have not chosen {STUDENT_ID, Name} as candidate key because {STUDENT_ID} alone can identify a unique row in the table and Name is redundant.

3. Primary Key

There can be more than one candidate key in relation out of which one can be chosen as the primary key. For Example, STUDENT_ID, as well as Aadhar both, are candidate keys for relation STUDENT but STUDENT_ID can be chosen as the primary key (only one out of many candidate keys).

- It is a unique key.
- It can identify only one tuple (a record) at a time.
- It has no duplicate values, it has unique values.
- It cannot be NULL.
- Primary keys are not necessarily to be a single column; more than one the column can also be a primary key for a table.
- A table cannot have more than one primary key.

4. Alternative Key

The candidate key other than the primary key is called an **alternate key**. For Example, STUDENT_ID, as well as Aadhar both, are candidate keys for relation STUDENT but Aadhar will be an alternate key (only one out of many candidate keys). It is a secondary key. All the keys which are not primary keys are called alternate keys.

3.6.2 Constraint on NULL Values

Another constraint on attributes specifies whether null values are or are not permitted. NULL values in relations may represent value unknown or value exists but not available. We can specify whether an attribute can have NULL or not using this constraint.

Example: If we do not want a NULL value for the student's name then we can specify and constraint it using NOT NULL.

Example : If phone number of a student is not mandatory then we we can specify and constraint it using NULL.

Scanned with OKEN Scanner

Definitions

Term	Definition
Key Constraints	Key Constraints are called uniqueness constraints since it ensures that every tuple in the relation should be unique.
Super Key	Superkey is an attribute or set of attributes which uniquely identifies each tuple in a relation.
Candidate Key	A Candidate key is a minimal super key with no redundant attributes (or) The minimal set of attributes that can uniquely identify a tuple is known as a candidate key.
Primary Key	A candidate key selected to uniquely identify all other attribute values in any given row. It cannot contain null entries.
Alternative Key	The candidate key other than the primary key is called an alternate key.

3.7 Entity Integrity, Referential Integrity and Foreign Keys

Entity Integrity

The entity integrity constraint states that no primary key value can be null. This is because the primary key value is used to identify individual tuples in a relation. Having null values for the primary key implies that, it may not be possible to identify some tuples.

Definition: Entity Integrity

Entity Integrity constraints says that no primary key can take NULL value, since using primary key we identify each tuple uniquely in a relation.

Example 1: If two or more tuples have null for their primary keys, then it would be hard to distinguish them, if they are referred from some other relations.

Referential Integrity

Referential Integrity Constraint ensures that there must always exist a valid relationship between two relational database tables. This valid relationship between the two tables confirms that a foreign key exists in a table. It should always reference a corresponding value or attribute in the other table or be null.

Definition: Referential Integrity

Referential integrity refers to the relationship between tables. The referential integrity constraint is specified between two relations and is used to maintain the consistency among tuples in the two relations. A referential integrity constraint is also known as foreign key constraint. A foreign key is a key whose values are derived from the Primary key of another table.

Scanned with OKEN Scanner

Example 2: Referential Integrity Constraint (RIC)

COURSE

course ID	name	credits	deptNo
CS635	ALGORITHMS	3	1
CS635	A.I	4	1
ES456	D.S.P	3	2
ME650	AERO DYNAMIC	3	3
CE751	MASS TRANSFER	3	4

DEPARTMENT

deptId	name	hod	phone
1	Computer Science	CS01	22576235
2	Electrical Engg.	ES01	22576234
3	Mechanical Engg.	ME01	22576233

The new courses refer to non-existent department and thus Violates RIC

Foreign Key

A foreign key is the one that is used to link two tables together via the primary key. It means the columns of one table points to the primary key attribute of the other table. It further means that if any attribute is set as a primary key attribute will work in another table as a foreign key attribute.

A Foreign key creates a relationship between two tables. The purpose of Foreign keys is to maintain data integrity and allow navigation between two different instances of an entity. It acts as a cross-reference between two tables as it references the primary key of another table.

Definition: Foreign Key

A foreign key is a set of attributes in a table that refers to the primary key of another table. In simple words, Foreign keys are the column of the table which is used to point to the primary key of another table.

To define referential integrity more formally, defining the concept of a *foreign key* is essential. The conditions for a foreign key, given below, specify a referential integrity constraint between the two relation schemas R_1 and R_2. A set of attributes FK in relation schema R_1 is a foreign key of R_1 that references relation R_2 if it satisfies the following two rules:

1. The attributes in FK have the same domain(s) as the primary key attributes PK of R_2; the attributes FK are said to reference or refer to the relation R_2.
2. A value of FK in a tuple t_1 of the current state r_1 (R_1) either occurs as a value of PK for some tuple t_2 in the current state r_2(R2) or is null. In the former case, we have $t_1[FK] = t_2[PK]$ and we say that the tuple t_1 references or refers to the tuple t_2.

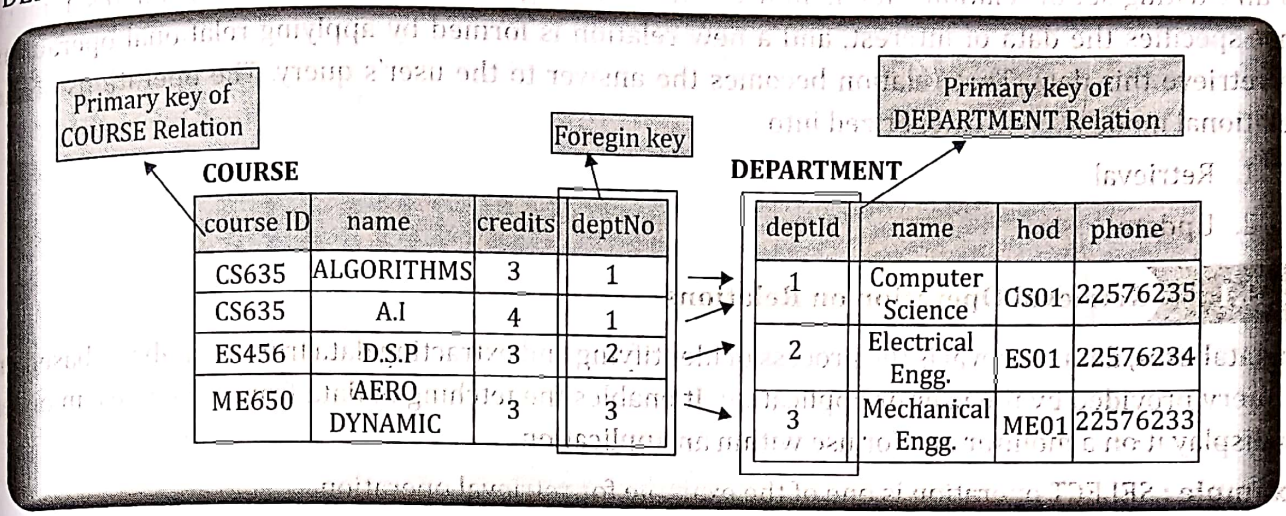
In this definition, R_1 is called the **referencing relation** and R_2 is the **referenced relation.**

If these two conditions hold, a referential integrity constraint from R_1 to R_2 is said to hold. In a database of many relations, there are usually many referential integrity constraints.

Scanned with OKEN Scanner

Example 3:

Foreign key attribute deptNo of COURSE relation refers to Primary key attribute deptID of DEPARTMENT relation Course.



course ID	name	credits	deptNo
CS635	ALGORITHMS	3	1
CS635	A.I	4	1
ES456	D.S.P	3	2
ME650	AERO DYNAMIC	3	3

deptId	name	hod	phone
1	Computer Science	CS01	22576235
2	Electrical Engg.	ES01	22576234
3	Mechanical Engg.	ME01	22576233

Example Foreign Key

3.8 Integrity Constraints Over Relations

A database is only as good as the information stored in it, and a DBMS must therefore help to prevent the entry of incorrect information. An **integrity constraint (IC)** is a condition that is specified on a database schema, and restricts the data that can be stored in an instance of the database. If a database instance satisfies all the integrity constraints specified on the database schema, it is a **legal** instance. A DBMS **enforces** integrity constraints, in that it permits only legal instances to be stored in the database. Integrity constraints are set of rules that can help to maintain the quality of information in the database. Integrity constraints are mostly used when trying to promote accuracy and consistency of data that is found in a relational database. This is very important to companies because information can be considered as an asset to certain organizations and it must be protected.

When dealing with integrity constraints there are a few sub-constraints which all tie back into the big picture of integrity constraints. There is referential integrity, entity integrity and domain integrity.

Definition: Domain Integrity

Domain integrity helps to ensure the length and type of data, the null value, the allowable value and the default value. When dealing with domain integrity, it ensures that the identity and purpose of the tables are clearly identified. Also, all fields must be consistent through the whole database; it should also clearly identify what modifications, comparisons and operations are allowed.

Definition: Integrity Constraints

Integrity Constraints are the set of constructs provided by a data model for specifying conditions that must be satisfied by the data. An Integrity constraint (IC) is a condition specified on a database schema and restricts the data that can be stored in an instance of the database.

Scanned with OKEN Scanner

3.9 Operations on Relations

A relational algebra expression forms a new relation after applying a number of algebraic operations to an existing set of relations; its main use is for querying a database. The user formulates a query that specifies the data of interest, and a new relation is formed by applying relational operations to retrieve this data. That relation becomes the answer to the user's query. The operations of the relational model can be categorized into

1. Retrieval
2. Update

3.9.1 Retrieval Operation on Relations

In databases, data retrieval is the process of identifying and extracting data from a database, based on a query provided by the user or application. It enables the fetching of data from a database in order to display it on a monitor and/or use within an application.

Example : SELECT operation is one of the example for retrieval operation.

3.9.2 Update Operation on Relations

There are three basic update operations on relations:

1. **Insert :** Insert is used to insert a new tuple or tuples in a relation,
2. **Delete :** Delete is used to delete tuples and
3. **Modify** (or Update) is used to change the values of some attributes in existing tuples.

Whenever these operations are applied, the integrity constraints specified on the relational database schema should not be violated. In this section, we discuss the different types of constraints that may be violated by each update operation and the types of actions that may be taken if an update does cause a violation.

Consider the relation **EMPLOYEE** and **DEPARTMENT**

EMPLOYEE

Eno	EName	DNo	EPhone	Salary
5010	Ashwini	10	9740185288	40,000
5020	Rashmi	20	9845698547	50,000
5030	Swapna	30	9449856972	60,000

DEPARTMENT

DNo	DName	DLocation
10	Computer science	Banashankari
20	Electronics	Banasavadi
30	Maths	J P Nagar

Foreignkey

The Insert Operation

The Insert operation provides a list of attribute values for a new tuple t that is to be inserted into a relation R. Insert can violate any of the four types of constraints as described below.

1. This insertion satisfies all constraints, so it is acceptable.

   Example: Insert <5010, "Ashwini", 10, 9740185288, 40,000> into Employee

Scanned with OKEN Scanner

2. Domain constraints can be violated if an attribute value is given that does not appear in the corresponding domain.

   Example: Insert <5010, "Ashwini", 10, 9710185288, 40,000, "**Java project**"> into Employee

   Is insertion operation is not possible because it violates domain constraint as it has entity "**Java project**" which does not corresponding any attribute in the original EMPLOYEE relation. This can be rectified by removing the extra entry in the new tuple and performing insert operation again.

3. Key constraints can be violated if a key value in the new tuple *t* already exists in another tuple in the relation *r(R)*.

   Example: Insert <5010, "Vidya", 10, 9845798660, 45,000> into Employee.

   - This insertion violates the key constraint because another tuple with the same ENo value already exists in the EMPLOYEE relation, and so it is rejected.

4. Entity integrity can be violated if the primary key of the new tuple *t* is null.

   Example: Insert < 'Anuradha', 20, 9758998743> into Employee.

   - This insertion violates the entity integrity constraint (null for the primary key ENo), so it is rejected.
   - In this operation, the DBMS could ask the user to provide a value for Eno and could accept the insertion if a valid Eno value was provided.

5. Referential integrity can be violated if the value of any foreign key in *t* refers to a tuple that does not exist in the referenced relation.

   Example: Insert <5040, "Vasudha", 40, 9448050836, 30,000> into EMPLOYEE.

   - This insertion operation violates the referential integrity constraint specified on DNo because the DNo = 40 of EMPLOYEE does not exist in the DEPARTMENT relation at all.

If an insertion violates one or more constraints, the default option is to *reject the insertion.* In this case, it would be useful if the DBMS could explain to the user why the insertion was rejected. Another option is to attempt to *correct the reason for rejecting the insertion,* but this is typically not used for violations caused by Insert; rather, it is used more often in correcting violations for Delete and Update.

The Delete Operation

The Delete operation is used to delete tuples from the relation R. The Delete operation can violate only referential integrity, if the tuple being deleted is referenced by the foreign keys from other tuples in the database. To specify deletion, a condition on the attributes of the relation selects the tuple (or tuples) to be deleted.

Example 1: Delete Operation

1. Delete the EMPLOYEE tuple with ENo = 5030 and DNo = 30.
   - This deletion is acceptable.
2. Delete the DEPARTMENT tuple with DNo = 20.
   - This deletion is not acceptable, because tuples in EMPLOYEE refer to this tuple. Hence, if the tuple is deleted, referential integrity violations will result.

Scanned with OKEN Scanner

Several options are available if a deletion operation causes a violation. Any one of the below options must be specified during database design for each foreign key constraint.

1. The first option is to ***reject** the **deletion.***
2. The second option is to *attempt* to ***cascade** (or propagate) the deletion* by deleting tuples that reference the tuple that is being deleted.
3. A third option is to *modify the referencing attribute values* that cause the violation; each such value is either **set to null** or changed to reference another valid tuple.

Notice that if a referencing attribute that causes a violation is part of the primary key, it cannot be set to null; otherwise, it would violate entity integrity. Combinations of these three options are also possible.

The Modify Operation

The Update (or Modify) operation is used to change the values of one or more attributes in a tuple (or tuples) of some relation R. It is necessary to specify a condition on the attributes of the relation to select the tuple (or tuples) to be modified.

- Updating an attribute that is neither a primary key nor a foreign key usually causes no problems; the DBMS need only check to confirm that the new value is of the correct data type and domain.
- Modifying a primary key value is similar to deleting one tuple and inserting another in its place, because we use the primary key to identify tuples.
- If a foreign key attribute is modified, the DBMS must make sure that the new value refers to an existing tuple in the referenced relation (or is null). Similar options exist to deal with referential integrity violations caused by Update as those options discussed for the Delete operation. In fact, when a referential integrity constraint is specified in the DDL, the DBMS will allow the user to choose separate options to deal with a violation Caused by Delete and a violation caused by Update.

Example 2: Update Operations

1. Update the SALARY of the EMPLOYEE tuple with ENo = 5010 to 60000.
   - Acceptable.
2. Update the DNO. of the EMPLOYEE tuple with ENo = 5030 to 10
   - Acceptable.
3. Update the DNO. of the EMPLOYEE tuple with ENo = 5030 to 40.
   - Unacceptable, because it violates referential integrity. i.e. DNo = 40 does not exits in the DEPARTMENT table.
4. Update the ENo. of the EMPLOYEE tuple with ENo = 5030 to 5010.
   - Unacceptable, because it violates primary key and referential integrity constraints. i.e The ENo. 5010 already exists, we cannot have two identical values for primary key.

Scanned with OKEN Scanner

3.10 Relational Algebra

A data model must include a set of operations to manipulate the database, in addition to the data model's concepts for defining database structure and constraints. The basic set of operations for the relational model is the relational algebra. These operations enable a user to specify basic retrieval requests. The result of retrieval is a new relation, which may have been formed from one or more relations. The algebra operations thus produce new relations, which can be further manipulated using operations of the same algebra. A sequence of relational algebra operations forms a relational algebra expression, whose result will also be a relation that represents the result of a database query (or retrieval request).

The relational algebra is very important for several reasons.

- First, it provides a formal foundation for relational model operations.
- Second, and perhaps more important, it is used as a basis for implementing and optimizing queries in relational database management systems (RDBMSs).

Definition: Relational Algebra

The relational algebra is a procedural query language. It consists of a set of operations that take one or two relations as input and produce a new relation as their result.

The relational algebra is often considered to be an integral part of the relational data model, and its operations can be divided into two groups.

- One group includes set operations from mathematical set theory; these are applicable because each relation is defined to be a set of tuples in the formal relational model. Set operations include
  - Union
  - Intersection
  - Set Difference, And
  - Cartesian Product.
- The other group consists of operations developed specifically for relational databases these include
  - Select
  - Project
  - Join

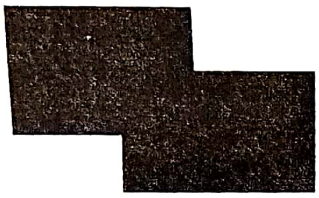
3.11 Relational Algebra Operations from Set Theory

Several set theoretic operations are used to merge the elements of two sets in various ways, including UNION, INTERSECTION, and SET DIFFERENCE (also called MINUS). These are binary operations; that is, each is applied to two sets (of tuples). When these operations are adapted to relational databases, the two relations on which any of these three operations are applied must have the same type of tuples; this condition has been called *union compatibility.* Two relations $R(A_1, A_2 \ldots, A_n)$ and $S(B_1, B_2, \ldots, B_n)$ are said to be union compatible if they have the same degree n and if domain (A_i) = domain (B_i) for $1 \leq i \leq n$. This means that the two relations have the same number of attributes, and each corresponding pair of attributes has the same domain.

Three operations UNION, INTERSECTION, and SET DIFFERENCE on two union-compatible relations R and S as follows:

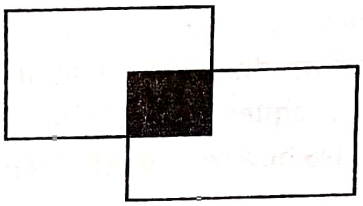
Scanned with OKEN Scanner

Union: The result of this operation, denoted by RUS, is a relation that includes all tuples that are either in R or in S or in both R and S. Duplicate tuples are eliminated.



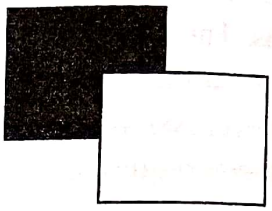
Union

Intersection: The result of this operation, denoted by R∩ S, is a relation that includes all tuples that are in both R and S.



Intersect

Set difference (or MINUS): The result of this operation, denoted by R - S, is a relation that includes all tuples that are in R but not in S.



Minus

Example **Consider the relation S_1 and S_2 on which Union, Intersection and Set Difference operations can be performed as follows**

S_1

Sid	Sname	Rating	Age
28	Ramesh	9	30
24	Ishwar	10	28
26	Mamatha	8	26
30	Vidya	9	28
32	Ashwini	9	25

S_2

Sid	Sname	Rating	Age
28	Ramesh	9	30
26	Mamatha	8	26
34	Sarala	9	28
50	Bharathi	10	30
32	Ashwini	9	25

$S_1 \cup S_2$ (Set union)

Sid	Sname	Rating	Age
28	Ramesh	9	30
24	Ishwar	10	28
26	Mamatha	8	26
30	Vidya	9	28
32	Ashwini	9	25
34	Sarala	9	28
50	Bharathi	10	30

$S_1 - S_2$ (Set difference)

Sid	Sname	Rating	Age
24	Ishwar	10	28
30	Vidya	9	28

$S_1 \cap S_2$ (Set intersection)

Sid	Sname	Rating	Age
28	Ramesh	9	30
26	Mamatha	8	26
32	Ashwini	9	25

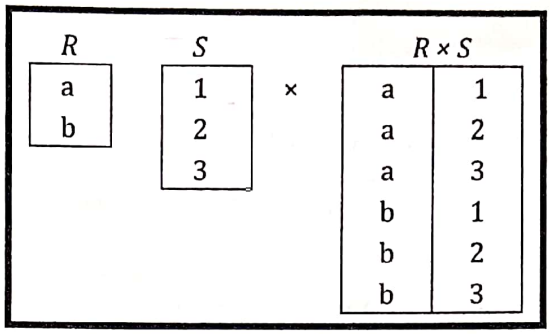
3.12 Binary Relational Operations

3.12.1 The Cartesian Product (Or Cross Product) Operation

The CARTESIAN PRODUCT operation-also known as CROSS PRODUCT or CROSS JOIN-which is denoted by x. This is also a binary set operation, but the relations on which it is applied do *not* have to be union compatible. This operation is used to combine tuples from two relations in a combinatorial

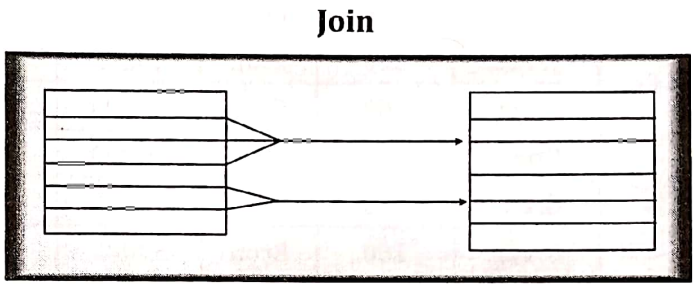
Scanned with OKEN Scanner

fashion. In general, the result of R(A_1, A_2, ..., A_n) X S(B_1, B_2, ..., B_m) is a relation Q with degree n + m attributes Q(A_1, A_2, ..., A_n B_1, B_2, ... , B_m), in that order. The resulting relation Q has one tuple for each combination of tuples-one from R and one from S. Hence, if R has nR tuples (denoted as IRI = nR), and S has nS tuples, then R x S will have $nR * nS$ tuples.



3.12.2 SQL: Join Operation: ⋈

- The *join* operation is one of the most useful operations in relational algebra and is the most commonly used way to combine information from two or more relations.
- Rows of two table are combined based on the given column(s) values. The tables being joined must have one common column.
- Join operations bring together two relations and combine their attributes and tuples in a specific fashion.



- The generic join operator (called the *Theta Join* is: ⋈)
- It takes arguments as the attributes from the two relations that are to be joined.
- The join condition can be = < > ≥ ≤ ≠
- When the join condition operator is = then we call this an *Equijoin*
- Note that the attributes in common are repeated.

Example 1:

Consider the EMP relation and a separate DEPART relation with (Dept, MainOffice, Phone) as follows

$$EMP \bowtie_{EMP.Dept = DEPART.Dept} DEPART$$

While different implementations have many ways of joining tables, in this section we concentrate on the most common types of join.

- Cross Join
- Equi Join
- Natural Join
- Non-Equi Join
- Outer Join
- Self Join

Example 2: Consider the two relations EMP and DEPART to illustrate all types of joins

Name	Office	Dept	Salary
Smith	400	CS	45000
Jones	220	Econ	35000
Green	160	Econ	50000
Brown	420	CS	65000

Dept	Main Office	Phone
CS	404	555-1212
Econ	200	555-1234
Fin	501	555-4321
Hist	100	555-9876

Scanned with OKEN Scanner

Cross Join

Each and every column of the first table will combine with each and every column of the second table.

Cross join operation will result in correct and incorrect values

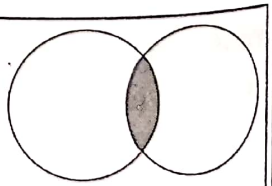
Name	Office	Dept	Salary	Dept	Main Office	Phone
Smith	400	CS	45000	CS	404	555-1212
Smith	400	CS	45000	Econ	200	555-1234
Smith	400	CS	45000	Fin	501	555-4321
Smith	400	CS	45000	Hist	100	555-9876
Jones	220	Econ	35000	CS	404	555-1212
Jones	220	Econ	Jones	Econ	200	555-1234
Jones	220	Econ	Jones	Fin	501	555-4321
Jones	220	Econ	Jones	Hist	100	555-9876
Green	160	Econ	50000	CS	404	555-1212
Green	160	Econ	50000	Econ	200	555-1234
Green	160	Econ	50000	Fin	501	555-4321
Green	160	Econ	50000	Hist	100	555-9876
Brown	420	CS	65000	CS	404	555-1212
Brown	420	CS	65000	Econ	200	555-1234
Brown	420	CS	65000	Fin	501	555-4321
Brown	420	CS	65000	Hist	100	555-9876

Equi Join

Perhaps the most used and important of the joins is the EQUIJOIN, also referred to as an INNER JOIN. The EQUIJOIN joins two tables with a common column in which each is usually the primary key.

Example 3: $EMP \bowtie_{emp.Dept = depart.Dept} DEPART$

Name	*Office*	*EMP.Dept*	*Salary*	*DEPART.Dept*	*MainOffice*	*Phone*
Smith	400	CS	45000	CS	404	555-1212
Jones	220	Econ	35000	Econ	200	555-1234
Green	160	Econ	50000	Econ	200	555-1234



Inner Join (result similar to Intersect)

Natural joins

A NATURAL JOIN is nearly the same as the EQUIJOIN; however, the NATURAL JOIN differs from the EQUIJOIN by eliminating duplicate columns in the joining columns. The JOIN condition is the same, but the columns selected differ. The natural join operator is (*).

Scanned with OKEN Scanner

Example 4: EMP*DEPART

Name	Office	Dept	Salary	MainOffice	Phone
Smith	400	CS	45000	404	555-1212
Jones	220	Econ	35000	200	555-1234
Green	160	Econ	50000	200	555-1234

Non-Equi Join

NON-EQUIJOIN joins two or more tables based on a specified column value not equaling a specified column value in another table.

Example 5: EMP ⋈emp.Dept != depart.Dept DEPART

Name	*Office*	*EMP.Dept*	*Salary*	*DEPART.Dept*	*MainOffice*	*Phone*
Smith	400	CS	45000	Econ	200	555-1234
Smith	400	CS	35000	Fin	501	555-4321
Smith	400	CS	50000	His	100	555-9876
Jones	220	Econ	35000	CS	404	555-1212
Jones	220	Econ	35000	Fin	501	555-4321
Jones	220	Econ	35000	Hist	100	555-9876
Green	160	Econ	50000	CS	404	555-1212
Green	160	Econ	50000	Fin	501	555-4321
Green	160	Econ	50000	Hist	100	555-9876
Brown	420	CS	65000	CS	404	555-1212
Brown	420	CS	65000	Econ	200	555-1234
Brown	420	CS	65000	Fin	501	555-4321
Brown	420	CS	65000	Hist	100	555-9876

Outer Join

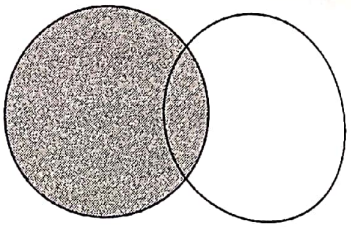
In the Join operations so far, only those tuples from both relations that satisfy the join condition are included in the output relation. The *Outer join* includes other tuples as well according to a few rules.

Three types of outer joins:

1. Left Outer Join ⟕ includes all tuples in the left hand relation and includes only those matching tuples from the right hand relation.
2. Right Outer Join ⟖ includes all tuples in the right hand relation and includes ony those matching tuples from the left hand relation.
3. Full Outer Join ⟗ includes all tuples in the left hand relation and from the right hand relation.
   - Left outer Join: EMP $\overset{}{\bowtie}_{\text{EMP.Dept.food = DEPART.Dept}}$ DEPART

Scanned with OKEN Scanner

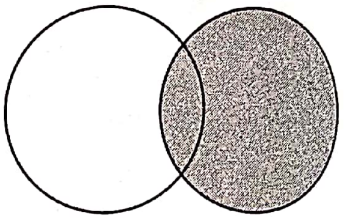
Name	Office	EMP. Dept	Salary	DEPART. Dept	MainOffice	Phone
Smith	400	CS	45000	CS	404	555-1212
Jones	220	Econ	35000	Econ	200	555-1234
Green	160	Econ	50000	Econ	200	555-1234
Brown	420	CS	65000	NULL	NULL	NULL
NULL	NULL	NULL	NULL	Fin	501	555-4321
NULL	NULL	NULL	NULL	Hist	100	555-9876



Left Outer Join

➤ Right outer Join :EMP $\bowtie_{\text{EMP.Dept = DEPART.Depat}}$ DEPART

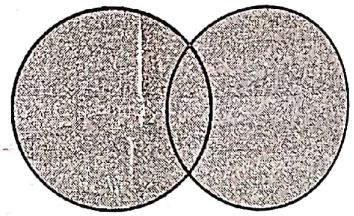
Name	Office	EMP. Dept	Salary	DEPART. Dept	MainOffice	Phone
Smith	400	CS	45000	CS	404	555-1212
Jones	220	Econ	35000	Econ	200	555-1234
Green	160	Econ	50000	Econ	200	555-1234
Brown	420	CS	65000	CS	404	555-1212



Right Outer Join

➤ Full Outer Join:EMP $\bowtie_{\text{EMP.Dept = DePART.Dept}}$ DEPART

Name	Office	EMP. Dept	Salary	DEPART. Dept	MainOffice	Phone
Smith	400	CS	45000	CS	404	555-1212
Jones	220	Econ	35000	Econ	200	555-1234
Green	160	Econ	50000	Econ	200	555-1234
Brown	420	CS	65000	NULL	NULL	NULL
NULL	NULL	NULL	NULL	Fin	501	555-4321
NULL	NULL	NULL	NULL	His	100	555-9876



Full Outer Join

Self Join

The SELF JOIN is used to join a table to itself, as if the table were two tables. With self joins it is quite simple. Self join is every other join type if both tables are the same.

3.12.3 Division Operation

The division operation is applied to two relations R(A) , S(B), where B subset A.

Let C = A – B (and hence A = X È Y); that is, let C be the set of attributes of R that are not attributes of S. For a tuple t to appear in the result T of the DIVISION, the values in t must appear in R in combination with *every* tuple in S. The division operator is a bit tricky to explain, and perhaps best approached through examples as will be done here.

Scanned with OKEN Scanner

Example

A

sno	pno
s1	p1
s1	p2
s1	p3
s1	p4
s2	p1
s2	p2
s3	p2
s4	p2
s4	p4

B1

pno
p2

B2

pno
p2
p4

B3

pno
p1
p2
p4

A/B1

sno
s1
s2
s3
s4

A/B2

sno
s1
s4

A/B3

sno
s1

Operations of Relational Algebra

Operation	Purpose	Notation
SELECT	Selects all tuples that satisfy the selction condition from a relation R.	$\sigma_{<\text{selection condtion}>}(R)$
PROJECT	Produces a new relation with only some of the attributes of R, and removes duplicate tuples	$\pi_{<\text{attrubute list}>}(R)$
THETA JOIN	Produces all combinations of tuples from R_1 and R_2 that satisfy the join condition.	$R_1 \bowtie_{<\text{join condtion}>} R_2$
EQUIJOIN	Produces all the combinations of tuples from R_1 and R_2 that satisfy a join condition with only equality comparisons.	$R_1 \bowtie_{<\text{join condition}>} R_2$, OR $R_1 \bowtie_{(<\text{join attributes 1}>)'\ (<\text{join attributes 2}>)} R_2$
NATURALI JOIN	Same as EQUIJOIN except that the join attributes of R_2 are not included in the resulting relation; if the join attributes have the same names, they do not have to be specified at all.	$R_1 *_{<\text{join condition}>} R_2$, OR $R_1 *_{(<\text{join attributes 1}>)'\ (<\text{join attributes 2}>)} R_2$, OR $R_1 * R_2$
UNION	Produces a relation that includes all the tuples in R_1 or R_2 or both R_1 and R_2; R_1 and R_2 must be union compatible	$R_1 \cup R_2$

Scanned with OKEN Scanner

INTERSECTION	Produces a relation that includes all the tuples in both R_1 and R_2; R_1 and R_2 must be union compatible.	$R_1 \cap R_2$
DIFFERENCE	Produces a relation that includes all the tuples in R_1 that are not in R_2; R_1 and R_2 must be union compatible	$R_1 - R_2$
CARTESIAN PRODUCT	Produces a relation that has the attributes of R_1 and R_2 and includes as tuples all possible combinations of tuples from R_1 and R_2	$R_1 \times R_2$
DIVISION	Produces a relation $R(X)$ that includes all tuples t(X) in R_1 that appear in R_1 in combination with every tuple from $R_2(Y)$, where $Z = X \cup Y$.	$R_1(X) \div R_2(Y)$

3.13 Additional Relational Operations

Some common database requests-which are needed in commercial query languages for **RDBMSs**-cannot be performed with the original relational algebra operations. Therefore some additional operations to express these requests. These operations enhance the expressive power of the original relational algebra.

3.13.1 Aggregate Functions and Grouping

Certain type of request that cannot be expressed in the basic relational algebra is to specify mathematical aggregate functions on collections of values from the database. **Aggregation function** takes a collection of values and returns a single value as a result. Examples of such functions include retrieving the average or total salary of all employees or the total number of employee tuples. These functions are used in simple statistical queries that summarize information from the database tuples. Common aggregate functions applied to attribute and tuples include

- Sum
- Maximum
- Count
- Average
- Minimum

Aggregate Function Examples

Consider the relation EMP has the following tuples:

Name	Office	Dept	Salary
Mithili	400	CS	45000
Ashwini	220	Econ	35000
Rashmi	160	Econ	50000
Swapna	420	CS	65000
Satish	500	Fin	60000

Scanned with OKEN Scanner

Example 1 : Find the minimum Salary: $\Im_{MIN\ (salary)}$ (EMP)	**MIN(salary)** 35000
Example 2 : Find the maximum Salary: $\Im_{max\ (salary)}$ (EMP)	**MAX(salary)** 65000
Example 3 : Find the average Salary: $\Im_{AVG\ (salary)}$ (EMP)	**AVG(salary)** 51000
Example 4 : Count the number of employees in the CS department: $\Im_{COUNT\ (name)}$ ($\sigma_{Dept = 'CS'}$ (EMP))	**COUNT(name)** 2
Example 5 : Find the total payroll for the Economics department: $\Im_{SUM\ (salary)}$ ($\sigma_{Dept = 'Econ'}$ (EMP))	**SUM(salary)** 85000

Another common type of request involves grouping the tuples in a relation by the value of some of their attributes and then applying an aggregate function independently to each group.

Recursive Closure Operations

Another type of operation that, in general, cannot be specified in the basic original relational algebra is recursive closure. This operation is applied to a recursive relationship between tuples of the same type.

Example

The relationship between an employee and a supervisor. This relationship is described by the foreign key SUPERSSN of the EMPLOYEE relation and it relates each employee tuple (in the role of supervisee) to another employee tuple (in the role of supervisor). An example of a recursive operation is to retrieve all supervisees of an employee e at all levels-that is, all employees e' directly supervised by e, all employees e" directly supervised by each employee e'; all employees e"' directly supervised by each employee e"; and so on.

EMPLOYEE

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO

3.14 Examples of Queries in Relational Algebra

In general the same queries can be stated in numerious ways using the various operations. Following are the additional examples to illustrate the different ways of querying in relational algebra.

Consider the relation schemas as follows.

works(person name, company name, salary);
lives(person name, street, city);
located in(company name, city);
managers(person name, manager name);

where manager name refers to person name.

Scanned with OKEN Scanner

(a) Find the names of the persons who work for company 'FBC' (company name = 'FBC').

Relational Algebra
result = $\pi_{\text{person_name}}(\sigma_{\text{company_name = 'FBC'}}(\text{works}))$

(b) List the names of the persons who work for company 'FBC' along with the cities they live in.

Relational Algebra
NamesForFBC = $\pi_{\text{person_name}}\sigma_{\text{company_name = 'FBC'}}(\text{works})$.
TheyLiveIn = NamesForFBC $\bowtie$ lives
result = $\pi_{\text{person_name, city}}$(TheyLiveIn).

(c) Find the persons who work for company 'FBC' with a salary of more than 10000. List the names of these persons along with the streets and cities where they live.

Relational Algebra
personsOfFBC = $\sigma_{\text{company_name = 'FBC'}}$(works).
NameWithBigsal = $\pi_{\text{person_name}}\sigma_{\text{salary > 10000}}$(PersonsOf FBC)
result = NameWithBigsal $\bowtie$ lives

(d) Find the names of the persons who live and work in the same city.

Relational Algebra
WorkLocation = $\pi_{\text{person_name, city}}$(works $\bowtie$ located_in).
result = $\pi_{\text{person_name}}\sigma_{\text{city = L.city}}$(WorkLocation × ρL (lives))

(e) Find the names of the persons who live in the same city and on the same street as their managers.

Relational Algebra
EAddMAdd = $\sigma_{\text{manager_name = M.person_name}}$($\rho$E (lives($\bowtie$ manager) × ρM(lives)
SameStreetCity = $\sigma_{\text{E.street = M.Street } \wedge \text{E. city = M.city}}$(EAddMAdd)
result = $\pi_{\text{E.person_name}}$(SameStreetCity)

(f) Find the names of the persons who do not work for company 'FBC'.

Relational Algebra
PersonForFBC = $\pi_{\text{person_name}}\sigma_{\text{company_name = 'FBC'}}$(works)
WorkPersons = $\pi_{\text{person_name}}$(works)
result = WorkPersons /- PersonForFBC

Scanned with OKEN Scanner

(g) Find the persons whose salaries are more than the salary of everybody who work for company 'SBC'.

Relational Algebra

$SalariesOfSBC = \pi_{salary}\, \sigma_{company_name = 'SBC'}(works)$

$WorkWithSalaryOfSBC = works \times \rho S(SalariesOfSBC)$

$EarnsLessThanSomeSBC = \pi_{person_name}\, \sigma_{salary \leq S.salary}(WorksWithSalaryOfSBC)$

$result = = \pi_{person_name}(works) - EarbsLessThanSomeSBC$

(h) Find the names of the companies that is located in every city where company 'SBC' is located in.

Relational Algebra

$ALLSBCCities = \pi_{city}\, \sigma_{company_name = 'SBC'}(located_in)$

$ImaginAllCompanyAtAllSBCLocations = \pi_{company_name}(located_in) \times AllSBCCities$

$NotReallyTrue = ImaginAllCompanyAtAllSBCLocations - located_in$

$result = \pi_{company_name}(located_in) - \pi_{company_name}(NotReallyTrue)$

Relational algebra (another solution):

$AllSBCCities = \pi_{city}\, \sigma_{company = 'SBC'}(located_in)$

$result = located_in / AllSBCCities$

3.15 Review Questions

Two Mark Questions and Answers

1. **Define the below terms.**
   - Attribute
   - Domain
   - Relational Schema
   - Degree of relation
   - Tuple
   - Relation Instance
   - Constraint
   - Cardinality of relation

Ans. **Domain:** A domain is a unique set of values permitted for an attribute in a table. Domain is atomic. A data type or format is also specified for each domain. It is possible for several attributes to have the same domain. The data type for Student_age is an integer number..

Attribute: Each column in a relation is called an attribute. The attributes are column heading in the table.

Tuple: The term tuple is often also called row and record in a database table. It is nothing but a single row of a table, which contains a single record.

Relation Instance: An instance of a relation is a set of tuples, also called records, in which each tuple has the same number of fields as the relation schema.

Relational Schema: The relation schema describes the column heads for the table. The schema specifies the relation's name, the name of each field (or column, or attribute), and the domain of each field.

Scanned with OKEN Scanner

Constraint: A property assigned to a table column that prevents certain types of invalid data values from being placed in the column.

Degree of Relation: The degree of a relation is the number of fields.

Cardinality of Relation: The cardinality of a relation instance is the number of tuples in it.

2. What is relational model and relational algebra?

Ans. Relational Model: The relational model represents a database model as a collection of relations. Each relation resembles a table of values.

Relational algebra: The relational algebra is a procedural query language. It consists of a set of operations that take one or two relations as input and produce a new relation as their result.

3. What is Domain Constraint?

Ans. Domain constraints in the schema specify an important condition that the user wants each instance of the relation to satisfy. Domain integrity helps to ensure the length and type of data, the null value, the allowable value and the default value.

4. What is Key Constraint and Super Key?

Ans. Key Constraints are called uniqueness constraints since it ensures that every tuple in the relation should be unique.

Superkey is an attribute or set of attributes which uniquely identifies each tuple in a relation.

5. What is Primary Key and Candidate Key?

Ans. Candidate Key : A Candidate key is a minimal super key with no redundant attributes (or) The minimal set of attributes that can uniquely identify a tuple is known as a candidate key.

Primary Key : A candidate key selected to uniquely identify all other attribute values in any given row. It cannot contain null entries.

6. Define Foreign key.

Ans. A foreign key is a set of attributes in a table that refers to the primary key of another table. In simple words, Foreign keys are the column of the table which is used to point to the primary key of another table.

7. What is relational algebra expression? Give an example.

Ans. A sequence of relational algebra operations forms a relational algebra expression, whose result will also be a relation that represents the result of a database query (or retrieval request).

8. What is Entity and Referential Integrity?

Ans. Entity Integrity constraints says that no primary key can take NULL value, since using primary key we identify each tuple uniquely in a relation.

Referential integrity refers to the relationship between tables. The referential integrity constraint is specified between two relations and is used to maintain the consistency among tuples in the two relations. A referential integrity constraint is also known as foreign key constraint. A foreign key is a key whose values are derived from the Primary key of another table.

Scanned with OKEN Scanner

9. What is Integrity Constraints?

Ans. Integrity Constraints are the set of constructs provided by a data model for specifying conditions that must be satisfied by the data. An Integrity constraint (IC) is a condition specified on a database schema and restricts the data that can be stored in an instance of the database.

II. Five Marks Questions

1. Explain Domain, Attribute, and Tuple with an example.
2. Describe the concept of relational model.
3. Discuss referential integrity.
4. Explain key constraints with examples.
5. Discuss Insert operation with example.
6. Discuss Delete operation with example.
7. Discuss Modify operation with example.
8. Discuss Project operation with example.
9. What is relational schema and relational instance? Give one example.
10. Explain Characteristics of Relation.
11. Explain the categories of Constraints.
12. Explain Domain Constraints with an example.
13. Explain UNION, INTERSECTION, and SET DIFFERENCE Operations.
14. Explain Cartesian Product (Or Cross Product) Operation.
15. Explain Natural Join with an example.
16. Explain Non-Equi Join with an example.
17. Explain Outer Join with an example.
18. Explain Division Operation with an example.
19. Explain Aggregate Functions with an example.
20. What is recursive closure operation? Explain with an example.

III. Eight Marks Questions

1. Explain different relational model constraint.
2. Explain SELECTION and PROJECTION with syntax and example.
3. Explain aggregate functions in relational algebra.
4. Discuss SET Operations with example.
5. Explain Unary Relational Operation.
6. Discuss Binary Relational Operation.

****

Scanned with OKEN Scanner